BSE3211 – Unit Testing Exercise: Test Plan

System: Meeting Planner (meetingplanner)

Group: GROUP L

Name	Student Number	Registration Number
Musheija Abraham	2300712139	23/U/12139/EVE
Lukaanya George	2300700696	23/U/0696
Kizito Ryan	2300710160	23/U/10160/EVE
Okema Paul Mark	2300716648	23/U/16648/EVE
Mwesigwa Sean Duncan	2100712248	21/U/12248/PS

---

1. Scope

The system manages calendars for employees and rooms, allowing:

- Booking meetings (with conflict detection)
- Booking vacation/all-day time blocks
- Checking availability for a room or person
- Printing the agenda for a room or person (by month or by day)

Testing is performed at the **unit level** using JUnit 4 on the following classes:

`Calendar`, `Meeting`, `Person`, `Room`, `Organization`

---

2. Test Approach

- Framework: JUnit 4 (`@Test`, `@Before`, `expected = ...`)
- Each test targets one behaviour (one reason to fail)
- Tests cover: normal inputs, boundary values, illegal inputs, and known seeded bugs
- For operations expected to throw, `TimeConflictException` type and message are verified

---

3. Test Cases

3.1 Calendar – `addMeeting`

ID	Description	Input	Expected
C01	Add a valid timed meeting	Month 3, Day 15, 09:00–10:00	No exception; slot marked busy
C02	Add a holiday (all-day, description only)	Month 2, Day 16, "Janan Luwum"	No exception; isBusy(2,16,0,23) = true

ID	Description	Input	Expected
C03	Add two non-overlapping meetings same day	09–10 then 11–12	No exception
C04	Add meeting whose start overlaps another	09–11 then 10–12	`TimeConflictException`
C05	Add meeting whose end overlaps another	10–12 then 08–11	`TimeConflictException`
C06	Add duplicate (same slot)	09–10 twice	`TimeConflictException`
C07	Day = 0 (illegal)	month 3, day 0	`TimeConflictException`
C08	Day = 32 (illegal)	month 3, day 32	`TimeConflictException`
C09	Month = 0 (illegal)	month 0	`TimeConflictException`
C10	Month = 13 (illegal)	month 13	`TimeConflictException`
C11	Negative start hour	start = -1	`TimeConflictException`
C12	End hour = 24	end = 24	`TimeConflictException`
C13	Start after end	start 12, end 9	`TimeConflictException`
C14	**December (month 12) — Bug 4**	month 12, day 1, 09–10	**No exception** (currently throws)
C15	**Same-hour start/end — Bug 5**	month 3, day 15, 09–09	**No exception** (currently throws)
C16	**November 30 bookable — Bug 3**	month 11, day 30, 09–10	**No exception** (currently throws)
C17	February 28 is valid	month 2, day 28, 09–10	No exception
C18	November 31 add (hack does not block via addMeeting)	month 11, day 31, 09–10	Should throw; currently succeeds — bug

3.2 Calendar – `isBusy`

ID	Description	Input	Expected
C19	Fresh calendar — no meetings	month 3, day 15, 09–10	false
C20	After adding a meeting	month 3, day 15, 09–10 (added)	true
C21	Invalid month	month 0	`TimeConflictException`
C22	**November 30 should be free — Bug 3**	month 11, day 30, 09–10 (fresh calendar)	**false** (currently true)
C23	November 31 always busy (correctly blocked)	month 11, day 31, 09–10	true

ID	Description	Input	Expected
C24	February 29 always busy (correctly blocked)	month 2, day 29, 09–10	true

3.3 Calendar – `checkTimes` (static)

ID	Description	Input	Expected
C25	**December valid — Bug 4**	month 12, day 1, 09–10	**No exception** (currently throws)
C26	**Same-hour valid — Bug 5**	month 3, day 15, 09–09	**No exception** (currently throws)

3.4 Calendar – `getMeeting` / `removeMeeting` / `clearSchedule`

ID	Description	Input	Expected
C27	getMeeting retrieves correct meeting	add then get index 0	correct Meeting object
C28	removeMeeting removes meeting	add then remove	slot no longer busy
C29	clearSchedule clears all meetings	add 2 meetings, clearSchedule	slot no longer busy
C30	**getMeeting invalid month — Bug 1**	month -1, day 15	`TimeConflictException` (currently throws `IndexOutOfBoundsException`)
C31	**removeMeeting invalid month — Bug 1**	month -1, day 15	`TimeConflictException` (currently throws `IndexOutOfBoundsException`)

3.5 Calendar – `printAgenda`

ID	Description	Input	Expected
C32	Print empty month	month 5 (no meetings)	Non-null string containing "5"
C33	Print empty day	month 3, day 15 (no meetings)	Non-null string
C34	Print day with full meeting	properly constructed Meeting	String contains meeting description

3.6 Meeting

ID	Description	Expected
M01	`Meeting(month, day)` constructor	start=0, end=23

ID	Description	Expected
M02	`Meeting(month, day, description)` constructor	start=0, end=23, description set
M03	`Meeting(month, day, start, end)` constructor	fields set correctly
M04	All getters and setters work	values round-trip correctly
M05	`toString()` with fully initialised Meeting	contains date, room ID, description
M06	`addAttendee()` on default-constructed Meeting	`NullPointerException` (attendees list not initialised) — bug

3.7 Person

ID	Description	Expected
P01	Add valid meeting	succeeds; isBusy returns true
P02	Add conflicting meeting	`TimeConflictException` with person's name in message
P03	isBusy on fresh person	false
P04	printAgenda (month)	non-null string
P05	getMeeting after add	correct Meeting returned
P06	removeMeeting after add	slot no longer busy

3.8 Room

ID	Description	Expected
R01	Add valid meeting	succeeds; isBusy returns true
R02	Add conflicting meeting	`TimeConflictException` with room ID in message
R03	isBusy on fresh room	false
R04	printAgenda (month) — empty month	non-null string

3.9 Organization

ID	Description	Expected
O01	getRoom with valid ID	correct Room object
O02	getRoom with invalid ID	Exception thrown
O03	getEmployee with valid name	correct Person object

ID	Description	Expected
O04	getEmployee with invalid name	Exception thrown
O05	getEmployees()	5 employees, non-null
O06	getRooms()	5 rooms, non-null

---

4. Known Seeded Bugs (from Lab PDF)

Bug	Location	Description	Test(s)
Bug 1	`Calendar.getMeeting`, `Calendar.removeMeeting`	No date/time validation — invalid inputs throw `IndexOutOfBoundsException` instead of `TimeConflictException`	C30, C31
Bug 2	`Calendar` constructor	Initialises months 0–13 (14 slots); month indices 0 and 13 are accessible	C09, C10
Bug 3	`Calendar` constructor	`occupied.get(11).get(30)` is blocked as "Day does not exist" — November 30 is a valid day	C16, C22
Bug 4	`Calendar.checkTimes`	Uses `mMonth >= 12` instead of `mMonth > 12` — December (month 12) is incorrectly rejected	C14, C25
Bug 5	`Calendar.checkTimes`	Uses `mStart >= mEnd` instead of `mStart > mEnd` — meetings with identical start and end hour are incorrectly rejected	C15, C26

Additional issues (not listed in lab)

- **Null description NPE in `addMeeting`:** `Calendar.addMeeting` calls `toCheck.getDescription().equals("Day does not exist")` without a null guard. If a previously added meeting has no description (e.g., created via `new Meeting(month, day, start, end)`), adding any second meeting on the same day throws `NullPointerException`. Fix: use `"Day does not exist".equals(toCheck.getDescription())` (null-safe comparison).
- `addMeeting` **silently accepts non-existent days:** The "Day does not exist" placeholder check inside `addMeeting` is skipped for those meetings, so a call like `addMeeting(new Meeting(11, 31, ...))` succeeds without throwing an exception (though `isBusy` correctly reports November 31 as always busy).
- `Meeting.addAttendee()` **NPE on default-constructed Meeting:** The `attendees` list is never initialised by the default constructor or `Meeting(month, day, start, end)`, so `addAttendee()` throws `NullPointerException` (M06).

• `Meeting.toString()` **NPE when** `room` **is null:** Any Meeting not constructed via the full 7-argument constructor has `room = null`; `toString()` calls `room.getID()` and throws `NullPointerException`. This also causes `Calendar.printAgenda` to NPE for any month containing placeholder or timed meetings without a room set.